

LES BASES DE LA PROGRAMMATION VHDL

Discipline :	Circuits Logiques et Programmables
Filiere :	Genie Informatique et Telecommunications (GIT) — Groupe 6
Etablissement :	Ecole Polytechnique d'Abomey-Calavi (EPAC) — Universite d'Abomey-Calavi (UAC)
Annee universitaire :	2025 - 2026
Superviseur :	Dr. Steaven CHEDE
Auteurs :	ADOME Hermann DEDJI Harry GBEDO Charbel HOUNZA Prisca
Document :	Support de cours approfondi — Toutes parties detaillees + Questions de preparation

SOMMAIRE

01	Introduction et Historique Origines, definition, contexte, FPGA vs ASIC, comparaison avec Verilog
02	Les Elements de Base du VHDL Lexique, librairies, entite, architecture, types, objets, sous-programmes
03	La Programmation VHDL Operateurs, PROCESS, instructions sequentielles, styles de description
04	Sujets Complementaires Importants Machines a etats, testbench, conception hierarchique, bonnes pratiques
05	Questions de Preparation a la Soutenance Questions theoriques, pratiques et de comprehension approfondies

PARTIE 01 — INTRODUCTION ET HISTORIQUE

1.1 — Qu'est-ce que le VHDL ?

Le **VHDL**, acronyme de **VHSIC Hardware Description Language** (ou **Langage de Description Matérielle VHSIC**), est un langage formel spécialement conçu pour **décrire des circuits électroniques numériques**. VHSIC lui-même signifie *Very High Speed Integrated Circuit* (Circuit Intégré à Très Grande Vitesse), ce qui reflète bien le contexte militaire et industriel dans lequel ce langage est né.

Il faut absolument comprendre ce qui distingue le VHDL des langages de programmation classiques comme le C, Python ou Java. Ces derniers décrivent une **suite d'instructions exécutées dans le temps** par un processeur. Le VHDL, lui, décrit une **structure matérielle** : des portes logiques, des registres, des connexions. Lorsqu'on écrit du VHDL, on ne programme pas un ordinateur — on **dessine un circuit électronique** de manière textuelle.

Cette description peut ensuite être utilisée à trois fins distinctes :

- **Modélisation** : décrire formellement la structure et les interfaces d'un circuit. Permet de documenter et de spécifier le comportement attendu.
- **Simulation** : vérifier virtuellement le fonctionnement du circuit sans le fabriquer physiquement. On applique des stimuli (entrées) et on observe les sorties simulées.
- **Synthèse** : à partir du code VHDL, un outil logiciel (synthétiseur) génère automatiquement le circuit physique correspondant, sous forme de portes logiques interconnectées.

Différence fondamentale : VHDL vs langage de programmation

Critère	Langage de programmation (C, Python)	VHDL
Nature	Description d'un algorithme	Description d'un circuit physique
Exécution	Séquentielle (une instruction après l'autre)	Parallèle (tout s'exécute simultanément)
Cible	Un processeur généraliste	FPGA, ASIC, circuit logique dédié
Temps	Notion de temps abstraite	Notion de temps physique (ns, ps)
Mémoire	Stockage dans des variables en RAM	Bascules, registres, éléments mémoire

1.2 — Contexte d'utilisation : FPGA et ASIC

Le VHDL est principalement utilisé pour deux types de circuits : les **FPGA** et les **ASIC**. Il est essentiel de comprendre la différence entre ces deux technologies.

Les FPGA (Field Programmable Gate Array)

Un FPGA est une puce électronique dont la logique interne peut être **configurée après fabrication**. Il contient un réseau de blocs logiques programmables interconnectés par des liaisons reconfigurables. En chargeant un fichier de configuration (généré par synthèse du code VHDL), on détermine quels blocs sont actifs et comment ils sont connectés, ce qui réalise physiquement le circuit décrit.

- **Avantages** : rapide à mettre en œuvre, reconfigurable à volonté, coûts de développement faibles.
- **Inconvénients** : plus lent et plus énergivore qu'un ASIC équivalent, coûteux en grande série.

- Exemples de fabricants : Xilinx (Vivado), Intel/Altera (Quartus), Lattice Semiconductor.
- Applications : prototypage rapide, traitement du signal (DSP), telecommunications, systemes embarques critiques.

Les ASIC (Application Specific Integrated Circuit)

Un ASIC est une puce concue et fabriquee **specifiquement pour une application donnee**. Contrairement au FPGA, sa logique est gravee de maniere permanente lors de la fabrication. Le VHDL est utilise pour specifier et verifier le circuit avant de le confier a un fondeur (ex : TSMC, Samsung Foundry).

- Avantages : performances optimales, consommation reduite, ideal pour la production en grande serie.
- Inconvenients : cout de developpement tres eleve, aucune correction possible apres fabrication.
- Exemples : processeur Apple M-series, puce Snapdragon de Qualcomm, circuit de cryptographie dedie.

1.3 — Historique detaille du VHDL

Annee	Evenement
1981	Le Departement de la Defense americain (DoD) lance le programme VHSIC. L'objectif est de creer un standard unique pour documenter et decire les circuits integres utilises dans les systemes militaires. IBM, Intermetrics et Texas Instruments sont selectionnes pour developper le langage.
1985	Publication de la version preliminaire du VHDL. Les premiers compilateurs experimentaux font leur apparition dans les laboratoires de recherche.
1987	Normalisation officielle par l'IEEE sous le numero IEEE Std 1076-1987 . Le VHDL devient un standard international. Cette version reste connue sous le nom VHDL-87.
1993	Premiere grande revision : IEEE Std 1076-1993 (VHDL-93) . Ameliorations syntaxiques, nouveaux types, meilleure gestion des attributs et des signaux.
1999	Extension VHDL-AMS (Analog and Mixed-Signal) , norme IEEE 1076.1, permettant de modeliser des circuits analogiques et mixtes (analogique + numerique).
2000 / 2002	Revisions mineures (VHDL-2000, VHDL-2002) corrigeant des ambiguites de la norme 1993 et ajoutant quelques fonctionnalites.
2008	Revision majeure : IEEE Std 1076-2008 (VHDL-2008) . Ajout de types generiques, amelioration du support des vecteurs, nouvelles instructions. Version tres utilisee aujourd'hui.
2019	Derniere revision : IEEE Std 1076-2019 . Clarifications et corrections mineures de la norme 2008.

1.4 — VHDL vs Verilog : comparaison

Le VHDL n'est pas le seul langage de description materielle. Son concurrent direct est **Verilog** (et son evolution SystemVerilog). Les deux sont des standards IEEE reconnus et utilises dans l'industrie.

Critere	VHDL	Verilog / SystemVerilog
Origine	DoD americain (1981)	Industrie privée (1984)
Typage	Fort (strict)	Faible (permissif)
Syntaxe	Verbeux, proche de l'ADA	Concis, proche du C
Popularite geographique	Europe, universites francophones	USA, Asie (industrie)
Simulation	Tres rigoureuse	Moins stricte
Utilisation courante	Enseignement, defense, Europe	ASIC design, industrie mondiale

NOTE : En France et en Afrique francophone, le VHDL est prefere dans les cursus universitaires. Les deux langages coexistent et certains projets utilisent meme les deux.

PARTIE 02 — LES ELEMENTS DE BASE DU VHDL

2.1 — Elements lexicaux

Comme tout langage formel, le VHDL possede ses propres regles d'ecriture. Les **elements lexicaux** sont les briques elementaires a partir desquelles on construit un programme. Le VHDL est **insensible a la casse** : Signal, SIGNAL et signal sont equivalents. En revanche, les **chaines de caracteres** et les **valeurs de bits** sont sensibles a la casse.

Les identificateurs

Un identificateur est un nom que le concepteur choisit pour designer une entite, un signal, un port, une variable, une constante, etc. Les regles sont strictes :

- Doit commencer par une lettre (A-Z ou a-z).
- Peut contenir des lettres, des chiffres (0-9) et le caractere de soulignement (_).
- Ne peut pas se terminer par un underscore.
- Ne peut pas contenir deux underscores consecutifs.
- Ne peut pas etre un mot reserve du langage.

Exemples valides : clk, data_in, compteur_8bits, FSM_etat

Exemples invalides : 8bits (commence par un chiffre), data__in (double underscore), signal (mot reserve)

Les mots reserves

Les mots reserves ont une signification fixee par la norme VHDL et ne peuvent pas etre utilises comme identificateurs. En voici les principaux :

Categorie	Mots reserves
Structure	LIBRARY, USE, PACKAGE, ENTITY, ARCHITECTURE, IS, BEGIN, END
Types	TYPE, SUBTYPE, ARRAY, RECORD, SIGNAL, VARIABLE, CONSTANT, FILE
Instructions seq.	PROCESS, IF, THEN, ELSIF, ELSE, END IF, CASE, WHEN, OTHERS, FOR, WHILE, LOOP, NEXT, EXIT
Instructions conc.	WITH, SELECT, GENERATE, COMPONENT, PORT MAP, GENERIC MAP
Logique	AND, OR, NOT, NAND, NOR, XOR, XNOR
Misc.	IN, OUT, INOUT, BUFFER, RETURN, FUNCTION, PROCEDURE, WAIT, AFTER, NULL

Les litteraux

Un litteral est une valeur constante ecrite directement dans le code. Il en existe plusieurs formes :

- **Entiers** : 0, 255, 1_024 (l'underscore est autorise pour la lisibilite)
- **Reels** : 3.14, 1.0E-3 (simulation uniquement)
- **Bit unique** : entre apostrophes — '0', '1', 'Z'
- **Vecteur de bits** : entre guillemets — "1010", "00001111"
- **Base numerique** : X"FF" (hexadecimal), O"17" (octal), B"11111111" (binaire)

- **Caracteres** : 'A', 'z' (type CHARACTER)
- **Chaines** : "hello" (type STRING)
- **Durees** : 10 ns, 5 us, 1 ms (simulation uniquement)

Les commentaires

En VHDL, les commentaires commencent par -- et s'etendent jusqu'a la fin de la ligne. Il n'existe pas de commentaires multi-lignes natifs en VHDL (contrairement au C avec /* ... */). Pour commenter plusieurs lignes, chaque ligne doit commencer par --.

```
-- Ceci est un commentaire sur une seule ligne
signal clk : STD_LOGIC; -- commentaire en fin de ligne
-- Pour plusieurs lignes, on repete -- sur chaque ligne :
-- Ligne 1 de commentaire
-- Ligne 2 de commentaire
```

2.2 — Structure generale d'un programme VHDL

Tout programme VHDL se compose de trois blocs principaux, toujours dans cet ordre : les **declarations de librairies**, l'**entite**, et l'**architecture**. La correspondance entre entite et architecture est obligatoire et se fait par le nom.

```
-- =====
-- BLOC 1 : Declarations de librairies
-- =====
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;
-- =====
-- BLOC 2 : Declaration de l'entite (interface externe)
-- =====
ENTITY nom_entite IS
    GENERIC (N : INTEGER := 8); -- parametres generiques (optionnel)
    PORT (
        clk      : IN  STD_LOGIC;
        reset    : IN  STD_LOGIC;
        entree    : IN  STD_LOGIC_VECTOR(N-1 DOWNT0 0);
        sortie    : OUT STD_LOGIC_VECTOR(N-1 DOWNT0 0)
    );
END nom_entite;
-- =====
-- BLOC 3 : Architecture (description interne du circuit)
-- =====
ARCHITECTURE comportementale OF nom_entite IS
    -- Zone de declaration : signaux internes, constantes...
    SIGNAL sig_inter : STD_LOGIC;
    CONSTANT PERIODE : TIME := 10 ns;
BEGIN
    -- Zone de description : instructions concurrentes et PROCESS
END comportementale;
```

NOTE : Une entite peut avoir **plusieurs architectures** associees, representant differents niveaux d'abstraction ou differentes implementations. La configuration VHDL permet de choisir laquelle utiliser.

2.3 — Librairies et Packages

Les **librairies** sont des espaces de noms contenant des definitions compilees (types, constantes, fonctions, etc.). Les **packages** sont des unites de compilation situees dans une librairie et regroupant des declarations partageables. La declaration `USE librairie.package.ALL` rend tous les elements du package visibles dans le fichier.

Les trois librairies standards

Librairie	Chargement	Contenu
ieee	Manuel (<code>LIBRARY ieee;</code>)	Packages de types et d'operations standards IEEE
std	Automatique (toujours disponible)	Types <code>BIT</code> , <code>BOOLEAN</code> , <code>INTEGER</code> , <code>REAL</code> , <code>CHARACTER</code> , <code>STRING</code> , <code>TIME</code>
work	Automatique (librairie de travail)	Tous les fichiers VHDL compiles de votre projet courant

Les packages IEEE essentiels

Package	Contenu principal	Recommandation
std_logic_1164	Types <code>STD_LOGIC</code> et <code>STD_LOGIC_VECTOR</code> (9 valeurs)	Indispensable — toujours inclure
numeric_std	Types <code>UNSIGNED</code> et <code>SIGNED</code> , operations arithmetiques modernes	Recommande (norme actuelle)
std_logic_arith	Operations arithmetiques sur <code>std_logic_vector</code>	Ancien — eviter dans nouveaux projets
std_logic_unsigned	Operations non signees sur <code>std_logic_vector</code>	Ancien — eviter dans nouveaux projets
std_logic_signed	Operations signees sur <code>std_logic_vector</code>	Ancien — eviter dans nouveaux projets
std_logic_textio	Lecture/ecriture de fichiers texte en simulation	Utilise dans les testbenches

ATTENTION : Ne pas utiliser simultanement `numeric_std` et `std_logic_arith` : ils definissent des fonctions en conflit. Preferer `numeric_std` seul.

Creer son propre package

Il est possible de creer ses propres packages pour centraliser les types, constantes et fonctions utilises dans plusieurs fichiers d'un projet. Un package se declare dans un fichier separe, puis son corps (optionnel, pour les sous-programmes) dans un autre bloc.

```
-- Fichier : mon_package.vhd
```

```

PACKAGE mon_package IS
  CONSTANT TAILLE_BUS : INTEGER := 8;
  TYPE etat_t IS (IDLE, ACTIF, ERREUR, FIN);
  FUNCTION max_int(a, b : INTEGER) RETURN INTEGER;
END PACKAGE mon_package;

-- Corps du package (implementation des fonctions)
PACKAGE BODY mon_package IS
  FUNCTION max_int(a, b : INTEGER) RETURN INTEGER IS
  BEGIN
    IF a > b THEN RETURN a; ELSE RETURN b; END IF;
  END FUNCTION;
END PACKAGE BODY mon_package;

-- Utilisation dans un autre fichier :
LIBRARY work;
USE work.mon_package.ALL;

```

2.4 — L'Entite (ENTITY)

L'entite est la description de l'**interface externe** d'un composant VHDL. Elle definit les **ports** (entrees et sorties), analogues aux broches d'un circuit integre physique. Conceptuellement, c'est une vue en "**boite noire**" : on sait ce qui entre et ce qui sort, mais on ne sait pas ce qui se passe a l'interieur (c'est le role de l'architecture).

En plus des ports, une entite peut contenir des **generiques** (GENERIC), qui sont des parametres de configuration permettant de rendre le composant generique (ex : definir la largeur d'un bus comme parametre plutot que de le figer).

```

ENTITY nom_entite IS
  GENERIC (
    -- Parametres de configuration (optionnel)
    N : INTEGER := 8;  -- largeur du bus, valeur par default = 8
    DELAI : TIME := 5 ns
  );
  PORT (
    -- Declaration des ports : nom : mode type;
    clk    : IN  STD_LOGIC;
    reset  : IN  STD_LOGIC;
    entree : IN  STD_LOGIC_VECTOR(N-1 DOWNT0 0);
    sortie : OUT STD_LOGIC_VECTOR(N-1 DOWNT0 0)
    -- Pas de point-virgule apres le DERNIER port !
  );
END [ENTITY] nom_entite;  -- ENTITY est optionnel apres END

```

Les modes de ports en detail

Mode	Sens du signal	Utilisation courante	Remarque importante
IN	Entree uniquement	Horloge, reset, donnees d'entree, adresse	Ne peut pas etre lu dans l'architecture (VHDL-93), peut etre lu depuis VHDL-2008

OUT	Sortie uniquement	Resultats, bus de sortie	Ne peut PAS etre relu dans l'architecture en VHDL-93. Utiliser BUFFER ou un signal interne.
INOUT	Bidirectionnel	Bus tristate (memoire, I2C, SPI partage)	Requiert une gestion de l'impedance avec 'Z'
BUFFER	Sortie avec retour interne	Compteur (la sortie est aussi une entree interne)	La sortie peut etre relue en interne, mais ne peut pas etre pilotee de l'exterieur facilement

Exemples d'entites

```
-- Exemple 1 : Porte logique AND simple
ENTITY porte_and IS
  PORT (a, b : IN STD_LOGIC; s : OUT STD_LOGIC);
END porte_and;

-- Exemple 2 : Bascule D synchrone avec reset asynchrone
ENTITY bascule_D IS
  PORT (
    clk   : IN  STD_LOGIC; -- horloge
    reset : IN  STD_LOGIC; -- reset asynchrone actif haut
    d     : IN  STD_LOGIC; -- entree donnee
    q     : OUT STD_LOGIC; -- sortie
    q_bar : OUT STD_LOGIC; -- sortie complementee
  );
END bascule_D;

-- Exemple 3 : Composant generique – registre N bits
ENTITY registre IS
  GENERIC (N : INTEGER := 8);
  PORT (
    clk   : IN  STD_LOGIC;
    reset : IN  STD_LOGIC;
    enable : IN  STD_LOGIC;
    d     : IN  STD_LOGIC_VECTOR(N-1 DOWNTO 0);
    q     : OUT STD_LOGIC_VECTOR(N-1 DOWNTO 0)
  );
END registre;
```

2.5 — L'Architecture (ARCHITECTURE)

L'architecture est indissociable de l'entite : elle decrit **comment fonctionne** le composant en interne. Elle comprend deux zones distinctes : la **zone de declaration** (avant BEGIN) et la **zone de description** (entre BEGIN et END).

Une meme entite peut avoir **plusieurs architectures**, representant des implementations differentes (comportementale, flot de donnees, structurelle). Cela permet par exemple de commencer par une description haut niveau pour simuler, puis de raffiner vers une description structurelle pour la synthese.

Contenu de la zone de declaration

- **SIGNAL** : signaux internes au composant (connexions entre sous-blocs).
- **VARIABLE** : ne peut etre declare ici — uniquement dans un PROCESS.
- **CONSTANT** : constantes locales a l'architecture.
- **COMPONENT** : declaration de composants utilises (style structurel).
- **TYPE / SUBTYPE** : definitions de types locaux.
- **FUNCTION / PROCEDURE** : sous-programmes locaux.
- **SHARED VARIABLE** : variable partagee entre plusieurs PROCESS (avancee).

La concurrence des instructions : concept fondamental

Le concept le plus important a assimiler en VHDL est la **concurrence**. Dans un programme C, les instructions s'executent une par une, dans l'ordre d'ecriture. En VHDL, toutes les instructions situees directement dans la zone de description d'une architecture (hors PROCESS) sont des **instructions concurrentes** : elles sont **toutes actives en permanence** et s'executent «en parallele».

Cela refilete la realite physique d'un circuit electronique : une porte AND continue de calculer sa sortie en permanence, independamment de ce que font les autres portes. Modifier l'ordre d'ecriture de deux instructions concurrentes ne change pas le resultat.

```
ARCHITECTURE demo OF circuit IS
  SIGNAL s1, s2, s3 : STD_LOGIC;
BEGIN
  -- Ces 3 instructions s'evaluent SIMULTANEMENT en permanence
  -- L'ordre d'ecriture ne change RIEN au resultat
  s3 <= s1 XOR s2; -- s'evalue des que s1 ou s2 change
  s1 <= a AND b;   -- s'evalue des que a ou b change
  s2 <= c OR d;    -- s'evalue des que c ou d change
  -- Meme si on ecrit s3 avant s1 et s2 !
END demo;
```

NOTE : Un PROCESS est lui-meme une instruction concurrente, mais son contenu interne est sequentiel. C'est le mecanisme qui permet de mixer sequentiel et parallele.

2.6 — Classes d'objets : SIGNAL, VARIABLE, CONSTANT

En VHDL, tout objet portant une valeur appartient a l'une des quatre classes suivantes. La distinction entre SIGNAL et VARIABLE est source d'erreurs frequentes chez les debutants.

Classe	Mot-cle	Operateur	Mise a jour	Portee	Utilisation
Signal	SIGNAL	<=	Differee (apres fin du PROCESS)	Architecture entiere (global)	Connexions entre composants, registres
Variable	VARIABLE	:=	Immediate (dans le PROCESS)	Local au PROCESS uniquement	Calculs intermediaires, boucles
Constante	CONSTANT	:=	Jamais (valeur figee)	Partout	Parametres fixes, tailles de bus

Fichier	FILE	—	—	Simulation	Lecture/écriture de fichiers testbench
---------	------	---	---	------------	--

Différence cruciale SIGNAL vs VARIABLE — exemple détaillé

```
-- === Comportement du SIGNAL ===
ARCHITECTURE avec_signal OF test IS
    SIGNAL s : INTEGER := 0;
BEGIN
    PROCESS(clk) BEGIN
        IF rising_edge(clk) THEN
            s <= s + 1; -- Mise à jour DIFFEREE
            s <= s + 1; -- La 2e affectation ECRASE la 1ere
            -- Resultat : s augmente de 1 à chaque cycle (pas de 2 !)
        END IF;
    END PROCESS;
END avec_signal;

-- === Comportement de la VARIABLE ===
ARCHITECTURE avec_variable OF test IS
BEGIN
    PROCESS(clk)
        VARIABLE v : INTEGER := 0; -- déclarée DANS le PROCESS
    BEGIN
        IF rising_edge(clk) THEN
            v := v + 1; -- Mise à jour IMMEDIATE
            v := v + 1; -- v a déjà été incrémenté !
            s <= v;      -- Resultat : s augmente de 2 à chaque cycle
        END IF;
    END PROCESS;
END avec_variable;
```

ATTENTION : Dans un **PROCESS** combinatoire, une variable non initialisée conserve sa valeur de l'itération précédente, ce qui peut créer un latch (mémoire involontaire) dans le circuit synthétisé. Toujours initialiser les variables avant usage.

2.7 — Les Types de données

Le VHDL est un langage **fortement typé** : chaque objet a un type précis, et les affectations entre types différents sont interdites sans conversion explicite. Cette rigueur évite des erreurs silencieuses qui seraient difficiles à déboguer sur circuit.

Types scalaires prédéfinis

Type	Valeurs possibles	Utilisation typique
BIT	'0', '1'	Logique simple, simulation uniquement
STD_LOGIC	'0','1','Z','X','U','W','L','H','-'	Logique standard industrielle (recommandée)
BOOLEAN	TRUE, FALSE	Conditions, generics de configuration

INTEGER	Au moins $-(2^{31})+1$ a $(2^{31})-1$	Compteurs, indices, calculs entiers
NATURAL	0 a $(2^{31})-1$ (sous-type de INTEGER)	Indices positifs, compteurs
POSITIVE	1 a $(2^{31})-1$ (sous-type de INTEGER)	Tailles, longueurs
REAL	Nombres reels (virgule flottante)	Simulation uniquement, modeles analogiques
TIME	Valeur + unite (ns, us, ms, s...)	Delais en simulation
CHARACTER	Caracteres ASCII	Messages de simulation
STRING	Tableau de CHARACTER	Messages d'assertions en simulation

STD_LOGIC : les 9 valeurs expliquees

Le type STD_LOGIC modelise avec precision les situations rencontrees dans un circuit reel. Ces 9 valeurs permettent de simuler des situations impossibles a representer avec un simple bit :

Valeur	Nom	Cas d'utilisation
'0'	Zero logique fort	Signal a 0 pilote activement par un driver fort
'1'	Un logique fort	Signal a 1 pilote activement par un driver fort
'Z'	Haute impendance	Signal deconnecte (bus tristate, port non pilote)
'X'	Inconnu fort	Conflit entre deux drivers (erreur de conception)
'U'	Non initialise	Signal non encore affecte au debut de simulation
'W'	Inconnu faible	Conflit entre signaux faibles
'L'	Zero faible (pull-down)	Signal a 0 via resistance de tirage vers le bas
'H'	Un faible (pull-up)	Signal a 1 via resistance de tirage vers le haut
'-'	Indifferent (don't care)	Optimisation : le synthetiseur peut choisir la valeur

Types composites : ARRAY et RECORD

```
-- ARRAY : tableau de valeurs de meme type
-- STD_LOGIC_VECTOR est en fait un ARRAY predefini :
-- TYPE STD_LOGIC_VECTOR IS ARRAY(NATURAL RANGE <>) OF STD_LOGIC;
-- Definir un tableau a indice descendant (convention hardware) :
SIGNAL bus_data : STD_LOGIC_VECTOR(7 DOWNT0 0);
-- Index 7 = bit de poids fort (MSB), Index 0 = bit de poids faible (LSB)
-- Definir un tableau a indice ascendant :
TYPE tableau_int IS ARRAY(0 TO 15) OF INTEGER;
```

```

SIGNAL memoire : tableau_int;

-- Tableau 2D (pour une memoire RAM) :
TYPE ram_t IS ARRAY(0 TO 255) OF STD_LOGIC_VECTOR(7 DOWNT0 0);
SIGNAL ram : ram_t;

-- RECORD : regroupe des champs de types differents
TYPE paquet_t IS RECORD
  adresse : STD_LOGIC_VECTOR(7 DOWNT0 0);
  donnee  : STD_LOGIC_VECTOR(15 DOWNT0 0);
  valide  : STD_LOGIC;
END RECORD;
SIGNAL paquet : paquet_t;
-- Acces a un champ : paquet.adresse <= X"1F";

```

Types enumeres — idéaux pour les machines a etats

```

-- Definition d'un type enumere
TYPE etat_machine IS (REPOS, DEMARRAGE, MARCHE, FREINAGE, ARRET, ERREUR);
SIGNAL etat_courant : etat_machine := REPOS;
SIGNAL etat_suivant : etat_machine;

-- Utilisation dans un CASE :
CASE etat_courant IS
  WHEN REPOS      => sortie <= '0'; IF start='1' THEN etat_suivant <= DEMARRAGE; END IF;
  WHEN DEMARRAGE  => sortie <= '1'; etat_suivant <= MARCHE;
  WHEN MARCHE     => sortie <= '1'; IF stop='1' THEN etat_suivant <= FREINAGE; END IF;
  WHEN FREINAGE   => sortie <= '0'; etat_suivant <= ARRET;
  WHEN ARRET      => sortie <= '0'; etat_suivant <= REPOS;
  WHEN OTHERS     => etat_suivant <= ERREUR;
END CASE;

```

Conversion de types : UNSIGNED, SIGNED et STD_LOGIC_VECTOR

En VHDL, STD_LOGIC_VECTOR, UNSIGNED et SIGNED sont des types distincts. Ils ont la meme representation binaire mais semantique differente. Des conversions explicites sont necessaires.

```

LIBRARY ieee;
USE ieee.numeric_std.ALL;

SIGNAL vec : STD_LOGIC_VECTOR(7 DOWNT0 0);
SIGNAL uns : UNSIGNED(7 DOWNT0 0);
SIGNAL sig : SIGNED(7 DOWNT0 0);
SIGNAL entier : INTEGER;

-- Conversions avec numeric_std :
uns    <= UNSIGNED(vec);           -- STD_LOGIC_VECTOR -> UNSIGNED
vec    <= STD_LOGIC_VECTOR(uns);   -- UNSIGNED -> STD_LOGIC_VECTOR
sig    <= SIGNED(vec);             -- STD_LOGIC_VECTOR -> SIGNED
entier <= TO_INTEGER(uns);          -- UNSIGNED -> INTEGER
uns    <= TO_UNSIGNED(entier, 8);   -- INTEGER -> UNSIGNED (8 bits)
sig    <= TO_SIGNED(entier, 8);     -- INTEGER -> SIGNED (8 bits)

```

2.8 — Les Sous-programmes : Fonctions et Procedures

Les sous-programmes permettent de factoriser du code réutilisable. En VHDL, il en existe deux types avec des rôles bien distincts :

Critère	Fonction (FUNCTION)	Procédure (PROCEDURE)
But principal	Calculer et retourner une valeur unique	Exécuter un algorithme complexe
Valeur de retour	Obligatoire via RETURN	Facultative via paramètres OUT/INOUT
Types de paramètres	IN uniquement	IN, OUT et INOUT
Utilisation	Dans une expression (ex : s <= max(a,b))	Appelée comme instruction indépendante
WAIT autorisé ?	Non	Oui (dans certains contextes)
Affectation signal	Non (retourne une valeur)	Oui (peut modifier des signaux)

```
-- === FONCTION : retourne une valeur ===
FUNCTION maximum(a, b : IN INTEGER) RETURN INTEGER IS
BEGIN
    IF a > b THEN RETURN a;
    ELSE RETURN b;
    END IF;
END FUNCTION maximum;

-- Utilisation :
resultat <= maximum(valeur_a, valeur_b);

-- === PROCEDURE : exécute un traitement ===
PROCEDURE reset_registre (
    SIGNAL clk      : IN  STD_LOGIC;
    SIGNAL reset    : IN  STD_LOGIC;
    SIGNAL reg      : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
) IS
BEGIN
    IF reset = '1' THEN
        reg <= (OTHERS => '0'); -- remet tous les bits à 0
    ELSIF rising_edge(clk) THEN
        NULL; -- rien à faire ici (exemple simplifié)
    END IF;
END PROCEDURE;
```

Surcharge (Overloading)

La surcharge permet de définir plusieurs sous-programmes portant le **même nom** mais avec des **types de paramètres différents**. Le compilateur choisit automatiquement la bonne version selon les types utilisés à l'appel. C'est ce mécanisme qui permet, par exemple, à l'opérateur + de fonctionner aussi bien avec des INTEGER qu'avec des UNSIGNED.

PARTIE 03 — LA PROGRAMMATION VHDL

3.1 — Les Operateurs en detail

Le VHDL dispose d'un ensemble complet d'operateurs, classes en familles. Il est important de connaitre leurs regles d'application et la priorite d'evaluation.

Operateurs logiques

Ces operateurs s'appliquent aux types BIT, STD_LOGIC, BOOLEAN, ainsi qu'aux vecteurs correspondants (STD_LOGIC_VECTOR, BIT_VECTOR). Appliques a des vecteurs, l'operation est effectuee **bit a bit**.

Operateur	Signification	Exemple scalaire	Exemple vectoriel
AND	ET logique	s <= a AND b	s <= "1010" AND "1100" -- => "1000"
OR	OU logique	s <= a OR b	s <= "1010" OR "1100" -- => "1110"
NOT	NON logique	s <= NOT a	s <= NOT "1010" -- => "0101"
NAND	NON-ET	s <= a NAND b	Inverse du AND bit a bit
NOR	NON-OU	s <= a NOR b	Inverse du OR bit a bit
XOR	OU exclusif	s <= a XOR b	s <= "1010" XOR "1100" -- => "0110"
XNOR	NON-OU exclusif	s <= a XNOR b	Inverse du XOR bit a bit

Operateurs arithmetiques et types NUMERIC_STD

Les operations arithmetiques (+, -, *, /) s'appliquent nativement aux types INTEGER, NATURAL et POSITIVE. Pour les operer sur des STD_LOGIC_VECTOR, il faut les convertir en UNSIGNED ou SIGNED (via numeric_std).

```
LIBRARY ieee;
USE ieee.numeric_std.ALL;

SIGNAL a, b : UNSIGNED(7 DOWNTO 0);
SIGNAL somme : UNSIGNED(8 DOWNTO 0); -- 9 bits pour eviter le debordement

-- Addition correcte avec debordement gere :
somme <= ('0' & a) + ('0' & b); -- extension de 1 bit + addition

-- Comparaison :
IF unsigned(a) > unsigned(b) THEN ...

-- Decalage (numeric_std 2008) :
a <= SHIFT_LEFT(a, 2); -- decalage gauche de 2 positions
a <= SHIFT_RIGHT(a, 1); -- decalage droite de 1 position
```

Priorite des operateurs (du plus au moins prioritaire)

Priorite	Operateurs
----------	------------

1 (plus haute)	** (puissance), ABS, NOT
2	*, /, MOD, REM
3	+ (unaire), - (unaire)
4	+ (binaire), - (binaire), & (concatenation)
5	SLL, SRL, SLA, SRA, ROL, ROR
6	=, /=, <, >, <=, >= (comparaisons)
7 (plus basse)	AND, OR, NAND, NOR, XOR, XNOR

ATTENTION : Les operateurs logiques (AND, OR, XOR...) ont la priorite LA PLUS BASSE. Il faut donc utiliser des parentheses pour combiner plusieurs conditions : ecrire (a AND b) OR c et non a AND b OR c.

Operateurs de decalage et de concatenation

```
SIGNAL vec : STD_LOGIC_VECTOR(7 DOWNTO 0) := "00001111";

-- Decalages (insere des '0') :
-- SLL : Shift Left Logical -> decale a gauche, insere '0' a droite
res <= vec SLL 2; -- "00001111" -> "00111100"

-- SRL : Shift Right Logical -> decale a droite, insere '0' a gauche
res <= vec SRL 3; -- "00001111" -> "00000001"

-- Rotations (bits qui sortent rentrent de l'autre cote) :
-- ROL : Rotate Left
res <= vec ROL 1; -- "00001111" -> "00011110"

-- ROR : Rotate Right
res <= vec ROR 2; -- "00001111" -> "11000011"

-- Concatenation & : assemble plusieurs vecteurs/bits
SIGNAL msb : STD_LOGIC_VECTOR(3 DOWNTO 0) := "1010";
SIGNAL lsb : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0101";
SIGNAL res16 : STD_LOGIC_VECTOR(7 DOWNTO 0);
res16 <= msb & lsb; -- => "10100101"

-- Exemple pratique : extension de signe
signal_etendu <= (others => '0') & signal_8bits; -- zero-extension
```

3.2 — Le PROCESS : coeur de la description sequentielle

Le PROCESS est l'instruction la plus importante du VHDL. C'est un bloc qui contient des **instructions sequentielles** (comme un programme C) mais qui est lui-meme une **instruction concurrente** de l'architecture. Plusieurs PROCESS coexistent et s'executent en parallele.

Syntaxe complete du PROCESS

```
[ label : ] PROCESS [ (liste_de_sensibilite) ]
-- Zone de declaration de VARIABLES (seulement ici)
VARIABLE var_locale : INTEGER := 0;
BEGIN
-- Instructions sequentielles : IF, CASE, LOOP, WAIT, affectations
```



```
-- Les affectations de SIGNAUX sont differees
-- Les affectations de VARIABLES sont immediates
END PROCESS [ label ];
```

La liste de sensibilité en détail

La liste de sensibilité contient les signaux dont un **changement de valeur** (evenement) déclenche l'exécution du PROCESS. Quand aucun des signaux de la liste ne change, le PROCESS est suspendu.

Règle pour les PROCESS combinatoires : tous les signaux lus dans le PROCESS doivent figurer dans la liste de sensibilité. Oublier un signal crée un latch involontaire.

Règle pour les PROCESS séquentiels : seuls l'horloge et le reset asynchrone (s'il existe) doivent figurer dans la liste de sensibilité.

```
-- PROCESS COMBINATOIRE : sorties = f(entrees actuelles seulement)
PROCESS(a, b, sel) -- TOUS les signaux lus doivent etre dans la liste
BEGIN
  IF sel = '0' THEN sortie <= a;
  ELSE sortie <= b;
  END IF;
END PROCESS;

-- PROCESS SEQUENTIEL : comportement depend de l'etat passe (basculer)
PROCESS(clk, reset) -- Seulement horloge et reset asynchrone
BEGIN
  IF reset = '1' THEN
    q <= '0'; -- reset asynchrone (prioritaire)
  ELSIF rising_edge(clk) THEN
    q <= d; -- capture sur front montant
  END IF;
END PROCESS;
```

Fonctions de détection de front

Pour détecter un front d'horloge, le VHDL fournit deux fonctions prédéfinies dans std_logic_1164 :

```
-- Front montant (0 -> 1) : le plus utilise
IF rising_edge(clk) THEN ...

-- Front descendant (1 -> 0) :
IF falling_edge(clk) THEN ...

-- Methode equivalente (plus ancienne, eviter) :
IF clk'EVENT AND clk = '1' THEN ...

-- clk'EVENT est un attribut qui indique qu'un evenement s'est produit sur clk
```

Reset synchrone vs Reset asynchrone

Type de reset	Caractéristique	Avantages	Inconvénients
---------------	-----------------	-----------	---------------

Asynchrone	Dans la liste de sensibilité. S'applique immédiatement, indépendamment de l'horloge.	Garantit l'initialisation même sans horloge. Recommande pour la mise sous tension.	Peut générer des métastabilités si mal synchronisé.
Synchrone	Évalue uniquement sur front d'horloge. Doit attendre le prochain cycle.	Plus simple à analyser statiquement (timing). Élimine les risques de glitch.	N'initialise pas le circuit sans horloge active.

```
-- Reset ASYNCHRONE (prioritaire, indépendant de l'horloge)
PROCESS(clk, reset)
BEGIN
  IF reset = '1' THEN          -- Vérifie en DEHORS du front
    q <= (OTHERS => '0');
  ELSIF rising_edge(clk) THEN
    q <= d;
  END IF;
END PROCESS;

-- Reset SYNCHRONE (applique uniquement sur front d'horloge)
PROCESS(clk)
BEGIN
  IF rising_edge(clk) THEN
    IF reset = '1' THEN        -- Vérifie DANS le front
      q <= (OTHERS => '0');
    ELSE
      q <= d;
    END IF;
  END IF;
END PROCESS;
```

3.3 — Instructions séquentielles

Instruction IF — comparaison avec d'autres langages

```
-- Syntaxe VHDL :
IF condition_1 THEN
  -- instructions si condition_1 vraie
ELSIF condition_2 THEN
  -- instructions si condition_2 vraie
ELSIF condition_3 THEN
  -- autant de ELSIF que nécessaire
ELSE
  -- instructions si toutes les conditions sont fausses
END IF; -- OBLIGATOIRE

-- Exemple : decodeur de priorité
PROCESS(a, b, c)
```

```

BEGIN
  sortie <= "00"; -- valeur par default (evite un latch)
  IF a = '1' THEN
    sortie <= "01"; -- a prioritaire
  ELSIF b = '1' THEN
    sortie <= "10";
  ELSIF c = '1' THEN
    sortie <= "11";
  END IF;
END PROCESS;

```

NOTE : Toujours assigner une valeur par default au debut d'un PROCESS combinatoire pour eviter la synthese de latches non desires.

Instruction CASE — pour les decodeurs et multiplexeurs

```

CASE expression IS
  WHEN valeur_1 => instructions;
  WHEN valeur_2 | valeur_3 => instructions; -- OU logique
  WHEN valeur_4 TO valeur_6 => instructions; -- plage (entiers)
  WHEN OTHERS => instructions; -- OBLIGATOIRE pour couvrir tous les cas
END CASE;

-- Exemple : multiplexeur 4 vers 1
PROCESS(sel, e0, e1, e2, e3)
BEGIN
  CASE sel IS
    WHEN "00" => sortie <= e0;
    WHEN "01" => sortie <= e1;
    WHEN "10" => sortie <= e2;
    WHEN "11" => sortie <= e3;
    WHEN OTHERS => sortie <= (OTHERS => 'X'); -- cas impossible
  END CASE;
END PROCESS;

-- Exemple : decodeur BCD -> 7 segments
CASE chiffre IS
  WHEN "0000" => seg <= "0111111"; -- 0
  WHEN "0001" => seg <= "0000110"; -- 1
  WHEN "0010" => seg <= "1011011"; -- 2
  WHEN OTHERS => seg <= "0000000"; -- eteint
END CASE;

```

Boucles FOR, WHILE et LOOP

```

-- Boucle FOR : nombre d'iterations fixe et connu a la compilation
-- Importante : la variable d'iteration est AUTOMATIQUE (pas besoin de la declarer)
FOR i IN 0 TO N-1 LOOP
  tab(i) <= (OTHERS => '0'); -- initialise N elements
END LOOP;

-- Boucle FOR descendante :
FOR i IN 7 DOWNT0 0 LOOP
  res(i) <= data(7 - i); -- inversion des bits

```

```

END LOOP;

-- Boucle WHILE : nombre d'iterations variable
-- ATTENTION : difficile a synthetiser, souvent reservee a la simulation
WHILE compteur < 256 LOOP
  compteur := compteur + 1;
END LOOP;

-- Boucle LOOP infinie avec EXIT conditionnel :
LOOP
  compteur := compteur + 1;
  EXIT WHEN compteur = 100; -- sort de la boucle
END LOOP;

-- NEXT : passe a l'iteration suivante (equivalent de 'continue' en C)
FOR i IN 0 TO 15 LOOP
  NEXT WHEN i = 5; -- saute l'iteration i=5
  tab(i) <= data;
END LOOP;

```

Instruction WAIT — pour les testbenches

L'instruction WAIT permet de suspendre l'execution d'un PROCESS jusqu'a une condition. Elle est principalement utilisee dans les **testbenches** (bancs de test). Un PROCESS ne peut pas avoir **a la fois** une liste de sensibilite et une instruction WAIT.

```

-- WAIT ON signal(s) : reprend execution quand un signal change
PROCESS
BEGIN
  WAIT ON clk, reset; -- attend un evenement sur clk OU reset
  IF reset = '1' THEN ...
END PROCESS;

-- WAIT UNTIL condition : reprend quand la condition devient vraie
PROCESS
BEGIN
  WAIT UNTIL rising_edge(clk); -- equivalent a la liste de sensibilite
  q <= d;
END PROCESS;

-- WAIT FOR duree : suspend pendant une duree (simulation seulement)
PROCESS -- Typique dans un testbench
BEGIN
  clk <= '0'; WAIT FOR 5 ns;
  clk <= '1'; WAIT FOR 5 ns; -- genere une horloge de periode 10 ns
END PROCESS;

-- WAIT sans condition : suspend indefiniment (fin du testbench)
PROCESS
BEGIN
  -- stimuli...
  WAIT; -- s'arrete ici pour ne pas reboucler
END PROCESS;

```

3.4 — Instructions concurrentes et styles de description

Hors des blocs PROCESS, toutes les instructions sont concurrentes. Il existe plusieurs formes d'instructions concurrentes :

Style Flot de donnees (Dataflow)

```
-- 1. Affectation simple :
sortie <= a AND b;          -- active des que a ou b change
s <= (a AND b) OR (c AND d);

-- 2. WHEN ELSE : assignation conditionnelle
sortie <= valeur_1 WHEN condition_1
      ELSE valeur_2 WHEN condition_2
      ELSE valeur_3 WHEN condition_3
      ELSE valeur_par_defaut;

-- Exemple : multiplexeur
sortie <= e0 WHEN sel = "00"
      ELSE e1 WHEN sel = "01"
      ELSE e2 WHEN sel = "10"
      ELSE e3;

-- 3. WITH SELECT : selection conditionnelle
WITH expression SELECT
  cible <= valeur_1 WHEN choix_1,
        valeur_2 WHEN choix_2,
        valeur_3 WHEN OTHERS;

-- Exemple : decodeur
WITH sel SELECT
  sortie <= e0 WHEN "00",
        e1 WHEN "01",
        e2 WHEN "10",
        e3 WHEN OTHERS;
```

NOTE : Difference entre WHEN ELSE et WITH SELECT : WHEN ELSE peut utiliser des conditions quelconques ($a > b$, $x = '1'$) tandis que WITH SELECT compare une seule expression a des valeurs specifiques.

Style Structurel — COMPONENT et PORT MAP

Le style structurel decrit le circuit comme une **interconnexion de composants**. Chaque composant est d'abord declare, puis instancie avec PORT MAP pour connecter ses ports. C'est l'equivalent d'un schema de cablage.

```
ARCHITECTURE struct OF circuit IS
  -- 1. Declaration des composants utilises
  COMPONENT porte_and IS
    PORT (a, b : IN STD_LOGIC; s : OUT STD_LOGIC);
  END COMPONENT;

  COMPONENT porte_or IS
    PORT (a, b : IN STD_LOGIC; s : OUT STD_LOGIC);
  END COMPONENT;

  -- 2. Signaux internes pour les connexions
  SIGNAL s1, s2 : STD_LOGIC;
BEGIN
```

```
-- 3. Instanciation et connexion (PORT MAP)
-- Connexion par nom (recommande) :
U1 : porte_and PORT MAP (a => e1, b => e2, s => s1);
U2 : porte_and PORT MAP (a => e3, b => e4, s => s2);
U3 : porte_or PORT MAP (a => s1, b => s2, s => sortie);

-- Connexion par position (deconseille, moins lisible) :
-- U1 : porte_and PORT MAP (e1, e2, s1);
END struct;
```

GENERATE : instanciation repetitive

L'instruction GENERATE permet de creer plusieurs instances d'un composant en une seule instruction, comme une boucle FOR concurrente. Tres utile pour les structures regulieres (bus, tableaux de cellules).

```
-- Exemple : creer 8 bascules D en parallele
ARCHITECTURE gen_reg OF registre_8 IS
  COMPONENT bascule_D IS
    PORT (clk, reset, d : IN STD_LOGIC; q : OUT STD_LOGIC);
  END COMPONENT;
BEGIN
  -- FOR GENERATE : instancie 8 bascules
  GEN_BASCULES : FOR i IN 0 TO 7 GENERATE
    BIT_i : bascule_D PORT MAP (
      clk    => clk,
      reset  => reset,
      d      => entree(i),
      q      => sortie(i)
    );
  END GENERATE;
END gen_reg;

-- IF GENERATE : instanciation conditionnelle
GEN_COND : IF enable_feature GENERATE
  U_extra : composant_optionnel PORT MAP (...);
END GENERATE;
```

PARTIE 04 — SUJETS COMPLEMENTAIRES IMPORTANTS

4.1 — Les Machines a Etats Finis (FSM)

Les **Machines a Etats Finis** (FSM — Finite State Machine) sont l'une des applications les plus importantes du VHDL. Elles permettent de modeliser tout systeme dont le comportement depend d'un etat interne (protocoles de communication, controleurs, sequenceurs).

Il en existe deux types principaux :

Type	Caracteristique	Sorties dependant de	Application typique
Moore	Les sorties dependent uniquement de l'etat courant	Etat seulement	Sequenceurs, signaux de controle stables
Mealy	Les sorties dependent de l'etat ET des entrees actuelles	Etat + entrees	Protocoles, reponses rapides (sans latence)

Architecture standard d'une FSM en VHDL (2 PROCESS)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY feu_tricolore IS
  PORT (
    clk      : IN  STD_LOGIC;
    reset    : IN  STD_LOGIC;
    rouge    : OUT STD_LOGIC;
    orange_l : OUT STD_LOGIC;
    vert     : OUT STD_LOGIC
  );
END feu_tricolore;

ARCHITECTURE fsm OF feu_tricolore IS
  TYPE etat_t IS (ST_ROUGE, ST_ROUGE_ORANGE, ST_VERT, ST_ORANGE);
  SIGNAL etat_courant : etat_t := ST_ROUGE;
  SIGNAL etat_suivant : etat_t;
  SIGNAL compteur      : INTEGER RANGE 0 TO 50 := 0;
BEGIN
  -- PROCESS 1 : registre d'etat (sequentiel)
  P_REGISTRE : PROCESS(clk, reset)
  BEGIN
    IF reset = '1' THEN
      etat_courant <= ST_ROUGE;
      compteur <= 0;
    ELSIF rising_edge(clk) THEN
      etat_courant <= etat_suivant;
      IF etat_courant /= etat_suivant THEN
        compteur <= 0; -- reset compteur au changement d'etat
      ELSE
        compteur <= compteur + 1;
      END IF;
    END IF;
  END PROCESS;
END fsm;
```

```

    END IF;
END IF;
END PROCESS;

-- PROCESS 2 : logique de transition et de sortie (combinatoire)
P_LOGIQUE : PROCESS(etat_courant, compteur)
BEGIN
    etat_suivant <= etat_courant; -- valeur par default
    rouge <= '0'; orange_l <= '0'; vert <= '0';
    CASE etat_courant IS
        WHEN ST_ROUGE =>
            rouge <= '1';
            IF compteur = 50 THEN etat_suivant <= ST_ROUGE_ORANGE; END IF;
        WHEN ST_ROUGE_ORANGE =>
            rouge <= '1'; orange_l <= '1';
            IF compteur = 5 THEN etat_suivant <= ST_VERT; END IF;
        WHEN ST_VERT =>
            vert <= '1';
            IF compteur = 40 THEN etat_suivant <= ST_ORANGE; END IF;
        WHEN ST_ORANGE =>
            orange_l <= '1';
            IF compteur = 5 THEN etat_suivant <= ST_ROUGE; END IF;
    END CASE;
END PROCESS;
END fsm;

```

4.2 — Le Testbench : simuler et verifier son circuit

Un **testbench** (banc de test) est un fichier VHDL special dont le seul but est de **simuler et verifier** un composant (appele DUT — Design Under Test). Un testbench n'est pas synthetisable : il peut utiliser des constructions interdites en synthese (WAIT FOR, types REAL, fonctions de simulation, etc.).

Sa structure typique : **entite vide** (pas de ports externes) + **architecture contenant** l'instanciation du DUT, la generation des signaux de test, et optionnellement des assertions automatiques.

```

-- Testbench pour la bascule_D
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY tb_basculer_D IS -- Entite VIDE (pas de ports)
END tb_basculer_D;

ARCHITECTURE sim OF tb_basculer_D IS
    -- Instanciation du composant a tester
    COMPONENT basculer_D IS
        PORT (clk, reset, d : IN STD_LOGIC; q : OUT STD_LOGIC);
    END COMPONENT;

    -- Signaux de test
    SIGNAL clk_tb : STD_LOGIC := '0';
    SIGNAL reset_tb : STD_LOGIC := '1';
    SIGNAL d_tb : STD_LOGIC := '0';
    SIGNAL q_tb : STD_LOGIC;

```



```

CONSTANT T_CLK : TIME := 10 ns;  -- periode horloge
BEGIN
  -- 1. Instancier le DUT
  DUT : bascule_D PORT MAP (
    clk => clk_tb, reset => reset_tb,
    d => d_tb, q => q_tb
  );

  -- 2. Generer l'horloge (PROCESS cyclique)
  P_CLK : PROCESS
  BEGIN
    clk_tb <= '0'; WAIT FOR T_CLK/2;
    clk_tb <= '1'; WAIT FOR T_CLK/2;
  END PROCESS;

  -- 3. Appliquer les stimuli
  P_STIM : PROCESS
  BEGIN
    reset_tb <= '1'; d_tb <= '0';
    WAIT FOR 25 ns;
    reset_tb <= '0';
    WAIT FOR 10 ns; d_tb <= '1';
    WAIT FOR 10 ns; d_tb <= '0';
    WAIT FOR 10 ns; d_tb <= '1';
    WAIT FOR 50 ns;
    ASSERT FALSE REPORT "Fin de simulation" SEVERITY FAILURE;
    WAIT;
  END PROCESS;

  -- 4. Verifications automatiques (assertions)
  P_CHECK : PROCESS
  BEGIN
    WAIT UNTIL rising_edge(clk_tb);
    WAIT FOR 1 ns;  -- attendre la stabilisation
    IF reset_tb = '0' THEN
      ASSERT q_tb = d_tb'DELAYED(T_CLK)
        REPORT "ERREUR : q ne suit pas d !" SEVERITY ERROR;
    END IF;
  END PROCESS;
END sim;

```

4.3 — Conception hierarchique et reutilisabilite

La **conception hierarchique** est une methode qui consiste a decomposer un systeme complexe en sous-systemes plus simples, chacun decrit par une entite/architecture independante. Ces sous-systemes sont ensuite assemblees via le style structurel.

Les **generics** rendent les composants parametrables, donc reutilisables dans differents contextes sans recrir de code. Par exemple, un additionneur generique sur N bits peut etre instancie en 8, 16 ou 32 bits simplement en changeant la valeur du generic.

```
-- Composant generique : additionneur N bits
```

```

ENTITY additionneur_gen IS
  GENERIC (N : INTEGER := 8);
  PORT (
    a, b : IN  UNSIGNED(N-1 DOWNT0 0);
    cin  : IN  STD_LOGIC;
    somme : OUT UNSIGNED(N-1 DOWNT0 0);
    cout : OUT STD_LOGIC
  );
END additionneur_gen;

-- Instanciation dans un niveau superieur :
U_ADD8 : additionneur_gen
  GENERIC MAP (N => 8) -- additionneur 8 bits
  PORT MAP (a => a8, b => b8, cin => '0', somme => s8, cout => c8);

U_ADD16 : additionneur_gen
  GENERIC MAP (N => 16) -- additionneur 16 bits
  PORT MAP (a => a16, b => b16, cin => '0', somme => s16, cout => c16);

```

4.4 — Bonnes pratiques de conception VHDL

- **Nommage explicite** : choisir des noms significatifs pour tous les signaux, ports et composants. Préférer compteur_8bits à cnt.
- **Un fichier par composant** : placer chaque paire entité/architecture dans un fichier séparé portant le nom du composant.
- **Valeurs par défaut dans les PROCESS combinatoires** : affecter toutes les sorties en début de PROCESS pour éviter les latches involontaires.
- **Éviter les boucles WHILE en synthèse** : utiliser FOR LOOP dont le nombre d'itérations est connu à la compilation.
- **Documenter avec des commentaires** : chaque entité, chaque PROCESS, chaque signal important doit avoir un commentaire explicatif.
- **Privilégier numeric_std** : éviter les anciens packages std_logic_arith et std_logic_unsigned au profit du standard actuel numeric_std.
- **Utiliser des constantes et des generics** : ne jamais coder en dur des valeurs numériques comme la taille d'un bus. Utiliser des CONSTANT ou des GENERIC.
- **Tester systématiquement** : écrire un testbench pour chaque composant avant de passer à la synthèse.

PARTIE 05 — QUESTIONS DE PREPARATION A LA SOUTENANCE

Cette section regroupe les questions les plus susceptibles d'être posées lors de la soutenance. Les réponses sont détaillées pour vous permettre de préparer vos explications. Il est conseillé de s'entraîner à répondre oralement, sans lire.

5.1 — Questions sur l'introduction et l'historique

Q1. Que signifie l'acronyme VHDL et que signifie VHSIC ?

VHDL signifie VHSIC Hardware Description Language. VHSIC signifie Very High Speed Integrated Circuit, c'est-à-dire Circuit Intégré à Très Grande Vitesse. Le projet VHSIC a été lancé en 1981 par le Département de la Défense des États-Unis dans le but de créer un standard unique pour la description et la documentation des circuits intégrés utilisés dans les systèmes militaires.

Q2. Quelle est la différence fondamentale entre le VHDL et un langage de programmation comme le C ?

Le C est un langage de programmation : il décrit une suite d'instructions exécutées séquentiellement par un processeur. Le VHDL est un langage de description matérielle : il décrit la structure et le comportement d'un circuit électronique physique. En VHDL, les instructions s'exécutent en parallèle (concurrency), ce qui reflète la réalité d'un circuit où tous les composants fonctionnent simultanément. De plus, le VHDL intègre une notion de temps physique (nanosecondes) indispensable pour la simulation.

Q3. Quelle est la différence entre un FPGA et un ASIC ? Dans quel cas choisir l'un ou l'autre ?

Un FPGA (Field Programmable Gate Array) est une puce reconfigurable après fabrication : on charge un fichier de configuration pour y implanter un circuit. Un ASIC (Application Specific Integrated Circuit) est une puce conçue et fabriquée définitivement pour une application unique. On choisit le FPGA pour le prototypage, les projets à faibles volumes ou les systèmes qui évoluent. On choisit l'ASIC en grande série car, malgré un coût de développement très élevé, il est plus performant et moins énergivore qu'un FPGA équivalent.

Q4. Quand le VHDL a-t-il été normalisé par l'IEEE et quel est le numéro de la norme ?

Le VHDL a été normalisé pour la première fois par l'IEEE en 1987 sous le numéro IEEE Std 1076-1987 (VHDL-87). Cette première normalisation a fait du VHDL un standard international reconnu. Il a ensuite été révisé en 1993 (VHDL-93), puis en 2008 (VHDL-2008), cette dernière version étant la plus utilisée aujourd'hui. La révision de 2019 est la plus récente.

Q5. Quelles sont les trois entreprises qui ont développé le VHDL à l'origine ?

Le développement du VHDL a été confié à trois acteurs majeurs de l'industrie informatique : IBM, spécialiste des systèmes informatiques ; Intermetrics, éditeur de logiciels de compilation ; et Texas Instruments, fabricant de semi-conducteurs. Cette collaboration a permis de créer un langage robuste qui couvre à la fois les aspects théoriques et pratiques de la conception électronique.

5.2 — Questions sur les éléments de base

Q6. Quelles sont les trois parties obligatoires d'un programme VHDL ? Quel est le rôle de chacune ?

Un programme VHDL se compose de trois parties : (1) les declarations de librairies (LIBRARY et USE), qui rendent accessibles les types et fonctions standards ; (2) l'entite (ENTITY), qui definit l'interface externe du composant, c'est-a-dire ses ports d'entree et de sortie ; (3) l'architecture (ARCHITECTURE), qui decrit le comportement interne du composant, c'est-a-dire comment les sorties sont calculees a partir des entrees.

Q7. Qu'est-ce que le package std_logic_1164 et pourquoi est-il indispensable ?

Le package std_logic_1164, situe dans la librairie ieee, definit le type STD_LOGIC et le type STD_LOGIC_VECTOR, qui sont les types de base de toute conception VHDL serieuse. Il est indispensable car le type BIT natif du VHDL n'a que deux valeurs ('0' et '1'), ce qui est insuffisant pour modeliser les situations reelles d'un circuit (haute impendance, valeur inconnue, non initialise). STD_LOGIC dispose de 9 valeurs pour couvrir tous ces cas.

Q8. Qu'est-ce que l'entite en VHDL ? Donnez un exemple concret.

L'entite definit la vue boite noire d'un composant : on voit uniquement ses entrees et ses sorties, pas son fonctionnement interne. C'est l'equivalent des broches d'un circuit integre physique. Par exemple, l'entite d'une porte AND a 2 entrees et 1 sortie declare : ENTITY porte_and IS PORT (a, b : IN STD_LOGIC; s : OUT STD_LOGIC); END porte_and; Ici, a et b sont les entrees de type STD_LOGIC, et s est la sortie. Rien n'indique encore que $s = a \text{ AND } b$: c'est le role de l'architecture.

Q9. Quels sont les 4 modes de ports en VHDL ? Citez un cas d'utilisation pour chacun.

Les 4 modes sont : IN (entree), utilise pour l'horloge, le reset, les donnees d'entree ; OUT (sortie), utilise pour les resultats de calcul, les bus de sortie ; INOUT (bidirectionnel), utilise pour les bus memoire partages ou les interfaces comme I2C ; BUFFER (sortie avec lecture interne), utilise quand une sortie doit aussi etre relue dans l'architecture, par exemple pour un compteur dont la valeur courante sert au calcul de la valeur suivante.

Q10. Quelle est la difference entre un SIGNAL et une VARIABLE en VHDL ?

La difference fondamentale porte sur le moment de mise a jour et la portee. Un SIGNAL est global a l'architecture et sa mise a jour est differee : l'affectation est prise en compte seulement apres la fin complete du PROCESS. Une VARIABLE est locale au PROCESS et sa mise a jour est immediate : la valeur change instantanement a la ligne suivante dans le PROCESS. Cela signifie que si on affecte deux fois un meme SIGNAL dans un PROCESS, seule la derniere affectation compte. Pour une VARIABLE, chaque affectation est prise en compte immediatement.

Q11. Pourquoi le type STD_LOGIC a-t-il 9 valeurs au lieu de 2 ? A quoi servent 'Z', 'X' et 'U' ?

Le type STD_LOGIC a 9 valeurs car un circuit reel peut se trouver dans des situations que '0' et '1' ne peuvent pas représenter. La valeur 'Z' (haute impendance) modelise un signal deconnecte, typiquement sur un bus tristate ou un port non pilote. La valeur 'X' (inconnue forte) apparait en simulation quand deux drivers pilotent simultanement un signal a des valeurs incompatibles, signalant un conflit dans la conception. La valeur 'U' (uninitialised) indique qu'un signal n'a jamais ete affecte, ce qui permet de detecter des oublis d'initialisation en simulation.

Q12. Quelle est la difference entre une fonction et une procedure en VHDL ?

Une fonction (FUNCTION) calcule et retourne obligatoirement une valeur via RETURN. Elle n'accepte que des parametres IN et ne peut pas modifier de signaux externes. Elle s'utilise dans une expression, comme $s \leq \text{maximum}(a, b)$. Une procedure (PROCEDURE) execute un traitement et peut avoir des parametres IN, OUT et INOUT. Elle peut modifier des signaux et ne retourne pas de valeur directement. Elle s'appelle comme une instruction independante. Les procedures sont plus flexibles mais les

fonctions sont plus naturelles pour les calculs.

5.3 — Questions sur la programmation VHDL

Q13. Qu'est-ce que la concurrence en VHDL ? Pourquoi est-ce une notion fondamentale ?

La concurrence signifie que toutes les instructions situees directement dans la zone de description d'une architecture s'executent simultanement et en permanence. C'est fondamental car cela reflete la realite physique d'un circuit electronique : une porte AND continue de calculer sa sortie independamment des autres composants, sans attendre. Contrairement au C ou Python, l'ordre d'ecriture des instructions concurrentes n'a aucune influence sur le resultat final. Un PROCESS est lui-meme une instruction concurrente, mais son contenu interne est sequentiel.

Q14. Qu'est-ce qu'un PROCESS ? Quand s'execute-t-il ?

Un PROCESS est une instruction concurrente de l'architecture qui contient du code sequentiel. Il s'execute quand au moins un des signaux de sa liste de sensibilite change de valeur. Entre deux evenements declencheurs, le PROCESS reste suspendu et inactif. Plusieurs PROCESS coexistent dans une architecture et s'executent en parallele. Un PROCESS peut etre soit combinatoire (sorties = fonction des entrees actuelles) soit sequentiel (sorties dependent de l'etat passe, avec detection d'un front d'horloge).

Q15. Que contient la liste de sensibilite d'un PROCESS combinatoire ? Et d'un PROCESS sequentiel ?

Pour un PROCESS combinatoire, la liste de sensibilite doit contenir TOUS les signaux qui sont lus dans le PROCESS. Oublier un signal cree un latch involontaire dans le circuit synthetise, ce qui est generalement une erreur de conception. Pour un PROCESS sequentiel (avec horloge), la liste contient uniquement l'horloge et le signal de reset si celui-ci est asynchrone. Les signaux de donnees (comme d dans une bascule D) n'y figurent pas car ils sont evalues uniquement sur le front d'horloge.

Q16. Quelle est la difference entre un reset synchrone et un reset asynchrone ?

Un reset asynchrone est place dans la liste de sensibilite du PROCESS et est evalue immediatement, independamment de l'horloge : des que reset passe a '1', le circuit se reinitialise instantanement, meme entre deux fronts d'horloge. Un reset synchrone n'est evalue que sur le front d'horloge : il faut attendre le prochain front pour que le reset soit pris en compte. Le reset asynchrone garantit une initialisation fiable meme si l'horloge n'est pas encore active (avantage au demarrage). Le reset synchrone est plus simple a analyser et evite les risques de glitch.

Q17. Quels sont les trois styles de description en VHDL ? Donnez les caracteristiques de chacun.

Le style comportemental (behavioral) utilise des PROCESS avec IF, CASE et LOOP pour decrire l'algorithme du circuit. Il est lisible et flexible, adapte aux circuits sequentiels complexes. Le style flot de donnees (dataflow) utilise des affectations concurrentes avec WHEN ELSE et WITH SELECT pour decrire comment les donnees circulent. Efficace pour les circuits combinatoires. Le style structurel utilise COMPONENT et PORT MAP pour assembler des composants existants. Il est equivalent a un schema de cablage et favorise la reutilisabilite. En pratique, on combine souvent les trois styles dans une meme architecture.

Q18. Quelle est la difference entre les instructions WHEN ELSE et WITH SELECT ?

Les deux sont des instructions concurrentes d'affectation conditionnelle, mais avec des differences. WHEN ELSE permet d'utiliser des conditions quelconques et potentiellement differentes pour chaque

branche ($a > b$, $x = '1'$, etc.). Les conditions sont évaluees dans l'ordre, la première vraie gagne. WITH SELECT compare une seule et même expression à des valeurs spécifiques, comme un CASE concurrent. WITH SELECT est préférable quand on sélectionne parmi des valeurs d'une même variable (comme un multiplexeur), car le compilateur peut optimiser la synthèse.

Q19. Qu'est-ce qu'un latch et comment l'éviter en VHDL ?

Un latch (ou verrou) est un élément mémoire combinatoire non désiré, créé par le synthétiseur quand une sortie n'est pas affectée dans tous les cas possibles d'un PROCESS combinatoire. Par exemple, dans un IF sans ELSE, la sortie garde sa valeur précédente si la condition est fausse, ce qui implique une mémoire. Pour l'éviter : toujours affecter une valeur par défaut à toutes les sorties au début du PROCESS, avant les IF et CASE. Ainsi, même si une branche n'est pas prise, la sortie a déjà une valeur définie.

Q20. Quelle est la priorité des opérateurs logiques par rapport aux opérateurs arithmétiques en VHDL ?

En VHDL, les opérateurs logiques (AND, OR, XOR, etc.) ont la priorité la PLUS BASSE de tous les opérateurs. Cela est surprenant pour quelqu'un habitué au C. Cela signifie que l'expression $a \text{ AND } b \text{ OR } c$ est évaluée comme $(a \text{ AND } b) \text{ OR } c$ uniquement si on utilise des parenthèses. Sans parenthèses, cette expression est en réalité ambiguë en VHDL (le compilateur peut générer une erreur si les priorités sont égales). La bonne pratique est toujours de parenthéser explicitement les expressions logiques complexes.

5.4 — Questions avancées et de réflexion

Q21. Pourquoi ne peut-on pas avoir à la fois une liste de sensibilité et une instruction WAIT dans un PROCESS ?

Ces deux mécanismes remplissent le même rôle : définir quand le PROCESS doit se réveiller. La liste de sensibilité est une forme implicite de WAIT ON (attendre un événement sur la liste). Les combiner créerait une ambiguïté pour le simulateur : doit-il se réveiller sur un événement de la liste, ou attendre la condition WAIT ? La norme VHDL interdit donc cette combinaison pour garantir un comportement de simulation déterministique et non ambigu.

Q22. Pourquoi l'ordre des instructions dans un PROCESS peut-il changer le résultat ?

À l'intérieur d'un PROCESS, les instructions s'exécutent séquentiellement, de haut en bas. Si on utilise des VARIABLES, le résultat dépend de l'ordre car les variables sont mises à jour immédiatement. En revanche, si on utilise uniquement des SIGNAUX, l'ordre n'a théoriquement pas d'importance pour le circuit final car toutes les affectations de signaux sont différées à la fin du PROCESS. Cependant, si on lit un signal dans le même PROCESS ou on l'a affecté, on lit l'ancienne valeur (avant mise à jour), ce qui peut induire en erreur.

Q23. Qu'est-ce qu'une machine à états de Moore et en quoi diffère-t-elle d'une machine de Mealy ?

Dans une machine de Moore, les sorties dépendent uniquement de l'état courant. Les sorties ne changent qu'au moment du changement d'état, ce qui les rend stables entre deux fronts d'horloge et plus faciles à synchroniser. Dans une machine de Mealy, les sorties dépendent à la fois de l'état courant ET des entrées actuelles. Les sorties peuvent donc changer immédiatement quand une entrée change, même sans changement d'état. La machine de Mealy répond plus rapidement mais ses sorties peuvent être instables (glitches) si les entrées changent entre deux fronts.

Q24. A quoi sert un testbench et pourquoi son entite est-elle toujours vide ?

Un testbench est un fichier VHDL conçu uniquement pour simuler et vérifier un composant. Il instancie le composant à tester (DUT), génère les signaux de stimulus (horloge, entrées), et peut vérifier automatiquement les sorties via des assertions. Son entité est vide (sans ports) car un testbench n'a pas d'interface avec l'extérieur : il représente un environnement de simulation autonome. Tous les signaux sont internes à son architecture. Un testbench n'est pas synthétisable et peut utiliser des constructions interdites en synthèse comme WAIT FOR.

Q25. Pourquoi utilise-t-on UNSIGNED ou SIGNED au lieu de STD_LOGIC_VECTOR pour les calculs arithmétiques ?

STD_LOGIC_VECTOR est simplement un tableau de bits sans interprétation arithmétique particulière. Le compilateur ne sait pas s'il doit l'interpréter comme un nombre non signé, signé, ou autre chose. UNSIGNED indique que le vecteur représente un entier positif (0 à $2^N - 1$). SIGNED indique qu'il représente un entier signé en complément à deux ($-2^{(N-1)}$ à $2^{(N-1)}-1$). Ces types permettent d'utiliser directement les opérateurs +, -, * et les comparaisons < et > avec une sémantique correcte. C'est pourquoi le package numeric_std est indispensable.

Q26. Qu'est-ce que la surcharge d'opérateurs (overloading) en VHDL et où est-elle utilisée ?

La surcharge permet de définir plusieurs versions d'un même opérateur ou sous-programme pour des types différents. Le compilateur choisit automatiquement la version correcte selon les types des opérandes. Par exemple, l'opérateur + est surchargé dans numeric_std pour fonctionner avec INTEGER, UNSIGNED et SIGNED. C'est grâce à la surcharge que l'on peut écrire `result <= a + b` que a et b soient des INTEGER ou des UNSIGNED, sans changer la syntaxe. Les concepteurs peuvent aussi définir leurs propres surcharges dans leurs packages.

Q27. Quelle est la différence entre DOWNTO et TO pour un vecteur ? Laquelle est recommandée en VHDL ?

Ces deux mots-clés définissent le sens d'indexation d'un vecteur. DOWNTO signifie que l'index le plus à gauche est le plus grand : `STD_LOGIC_VECTOR(7 DOWNTO 0)` a l'index 7 comme bit de poids fort (MSB) et l'index 0 comme bit de poids faible (LSB), ce qui est la convention standard en électronique. TO signifie que l'index le plus à gauche est le plus petit : `STD_LOGIC_VECTOR(0 TO 7)` a l'index 0 à gauche. La convention DOWNTO est fortement recommandée en VHDL car elle correspond à la représentation binaire habituelle et évite les confusions de bits lors des concatenations.

Q28. Que signifie l'instruction NULL en VHDL ? Dans quel contexte est-elle utilisée ?

L'instruction NULL est une instruction qui ne fait rien. Elle est utilisée dans un CASE pour indiquer explicitement qu'aucune action n'est requise pour certains cas, et ce de manière intentionnelle. Par exemple : `WHEN OTHERS => NULL`; dans un CASE indique au lecteur et au compilateur que les cas non énumérés sont délibérément ignorés, et non oubliés par erreur. Sans NULL, certains compilateurs ou outils d'analyse pourraient générer des avertissements pour les branches sans instruction.

Q29. Comment générer une horloge dans un testbench ? Donnez la syntaxe VHDL.

Dans un testbench, on génère une horloge avec un PROCESS cyclique utilisant WAIT FOR. La syntaxe classique est : `PROCESS BEGIN clk <= '0'; WAIT FOR 5 ns; clk <= '1'; WAIT FOR 5 ns; END PROCESS`; Ce PROCESS ne possède pas de liste de sensibilité et ne contient pas de WAIT final, donc il se répète indéfiniment, générant une horloge de période 10 ns et de rapport cyclique 50%. On peut utiliser une CONSTANT pour la période : `CONSTANT T_CLK : TIME := 10 ns; puis clk <= NOT clk AFTER T_CLK/2`;

Q30. Quelles sont les caracteristiques qui rendent un code VHDL 'non synthetisable' ?

Un code VHDL non synthetisable contient des constructions comprises par le simulateur mais impossible a realiser en circuit physique. Les principales sont : les instructions WAIT FOR duree (le temps n'existe pas en synthese) ; les types REAL (pas de virgule flottante en logique numerique) ; les boucles WHILE dont le nombre d'iterations n'est pas connu a la compilation ; les initialisations de signaux avec AFTER (ex : `s <= '1' AFTER 5 ns`) ; la lecture/ecriture de fichiers ; et les affectations multiples non conditionnelles a un meme signal depuis plusieurs PROCESS (court-circuit logique).

Bonne chance pour votre soutenance !

ADOME Hermann | DEDJI Harry | GBEDO Charbel | HOUNZA Prisca

EPAC / UAC — Filiere GIT3 — Annee 2025-2026

Sous la supervision de Dr. Steaven CHEDE